Graph-Oriented Generation (GOG): Pre-Generation Repository Orientation for Agentic Coding Assistants

Derek R. Chisholm

Brandeis University / Independent Researcher

dchisholm125@gmail.com · https://derekchisholm.com · https://github.com/dchisholm125

Abstract

Agentic coding assistants navigate software repositories through exploration: reading files, searching symbols, and following import chains before generating code. This exploration is expensive — measured in tool calls, transcript tokens, and the risk of navigating toward structurally wrong but semantically plausible files. We present Graph-Oriented Generation (GOG), a repository intelligence layer that orients a coding assistant before generation begins. GOG onboards a repository into persistent symbolic state, selects a context strategy from structural signals, classifies every file as an edit target or evidence-only surface using deterministic path analysis, and delivers a compact structured brief to the assistant prior to invocation. In a controlled benchmark against the Orca Whirlpools SDK — a production TypeScript and Rust repository — an assistant given a GOG brief consumed 14 tool calls and 196,008 transcript tokens to produce a result that passed both runtime tests and static typecheck. The same assistant with no brief consumed 38 tool calls and 626,497 tokens, and failed static typecheck. A hybrid RAG baseline also passed, using 20 tool calls and 471,757 tokens. GOG's edit boundary classification is fully automated; natural-language hazard synthesis from graph evidence remains an active research target. All results are from a single repository and should be read as early evidence for the orientation hypothesis, not a generalization claim. GOG Lite, the public reference implementation, is available at https://github.com/dchisholm125/graph-oriented-generation.

1. Introduction

Every time a coding assistant begins working on a repository it has never seen, it faces the same problem: it must discover the codebase before it can help with it. Which files are relevant? Which are generated and should not be edited? Where do the package boundaries lie? Which type is the correct public API surface and which is an internal implementation detail? Without answers to these questions, the assistant explores — reading files, searching symbols, following import chains — before it can write a single line of code.

This exploration is not free. In agentic coding systems, exploration manifests as tool calls: file reads, directory listings, symbol searches, and grep queries. Each tool call consumes tokens, adds latency, and introduces an opportunity for the model to navigate toward the wrong part of the codebase. When the model explores and reaches a plausible-but-wrong file, the resulting code may compile, may pass runtime tests, and may still be incorrect — because the error is semantic, not syntactic. Static typecheckers catch it later. CI catches it later. The developer catches it later.

This paper asks a simple question: what if the assistant were oriented before generation begins?

We present Graph-Oriented Generation (GOG), a repository intelligence layer that operates before a coding assistant begins generating. GOG onboards a software repository into persistent symbolic state — a structural graph encoding files, packages, imports, symbols, generated surfaces, and architectural strata. When a task arrives, GOG selects a context strategy, identifies edit and evidence boundaries, classifies semantic hazards, and delivers a compact structured brief to the assistant before generation begins. The assistant arrives knowing where it is, what it is allowed to edit, and which paths lead to known failure modes.

The effects of this orientation are measurable. In a controlled benchmark against the Orca Whirlpools SDK — a production TypeScript and Rust repository — a coding assistant given a GOG brief consumed 14 tool calls to produce a validated result. The same assistant given no brief consumed 38 tool calls and produced a result that failed static typecheck. The model did not reason differently; it navigated differently. It spent fewer tokens discovering what GOG had already determined.

GOG is not a coding assistant and not a retrieval system. It is infrastructure for coding assistants: the layer that transforms a repository from an undifferentiated directory into an oriented symbolic substrate. Its closest intellectual ancestors are static analysis tools and language server protocols — systems that understand code structure deterministically — but GOG adds a dimension those tools lack: the ability to translate structural knowledge into natural-language orientation for a language model about to generate.

This work grows from a broader theoretical research program, the Symbolic Reasoning Membrane (SRM), which investigates the separation of symbolic reasoning from linguistic generation in AI systems. GOG is the applied instantiation of that thesis in the software repository domain: the repository is the structured domain, GOG is the symbolic layer that reasons over it, and the coding assistant is the linguistic renderer that produces syntax from oriented context. The SRM theoretical foundations and empirical validation are treated in a companion paper; this paper focuses on the GOG system and its measured effects on coding assistant behavior.

The contributions of this paper are:

- **A five-stage orientation pipeline** for software repositories, from onboarding through brief delivery, with a context strategy decision layer that selects among five serving modes based on structural repository signals.
- **An automated file edit policy classifier** that distinguishes edit targets from evidence-only files using path analysis and file header inspection, with no per-task configuration.
- **A native coding assistant integration** (opencode-gog) demonstrating GOG as a preflight layer beneath an existing agentic coding system, with no changes to the assistant's internal architecture.
- **Controlled benchmark evidence** across three layers of experimental reproducibility showing that pre-generation orientation reduces tool call count, transcript token consumption, and wrong-stratum file reads on a real production repository with executable validation.
- **An honest characterization** of the boundary between automated structural classification (currently implemented) and automated natural-language hazard synthesis (an active research target), with a proposed regression methodology for validating generalization.

2. Related Work

The problem of providing relevant context to a language model generating code over a large repository has attracted substantial research attention. We situate GOG within this landscape and describe the specific gap it addresses.

2.1 Retrieval-Augmented Generation for Code

Standard RAG pipelines for code embed repository files into a vector index and retrieve semantically similar chunks in response to a prompt (Gao et al., 2023). Applied to software repositories, this approach has a structural limitation: vector similarity captures semantic proximity, not architectural proximity. A file may be semantically similar to a query — it shares vocabulary, symbol names, or conceptual territory — while being structurally irrelevant to the edit. Conversely, the correct edit surface may use vocabulary distant from the query while being the only structurally valid target.

The distinction matters most at the boundary between generated and handwritten code, between public API surfaces and internal implementation details, and between packages in a cross-language repository. These boundaries are properties of the repository's architecture, not of the semantic content of individual files. Retrieval systems that operate on content similarity cannot reliably respect them.

GOG's approach differs in a specific way: it does not replace retrieval with graph traversal. The hybrid RAG baseline used in this paper's benchmarks combines vector retrieval with graph-assisted candidate generation, and it passes the same benchmark tasks that GOG passes. The difference is in what each system provides to the assistant: RAG provides a ranked list of files; GOG provides a ranked list of files plus explicit edit boundaries, structural anchors, and hazard guidance derived from the repository's onboarded symbolic state. That additional structure is what produces the tool call reduction reported in Section 4.

2.2 Graph-Based Retrieval

Microsoft's GraphRAG (Edge et al., 2024) constructs knowledge graphs from document collections using an LLM to extract entities and relationships, then uses community detection over the graph to support global queries that naive RAG cannot answer. GraphRAG addresses a genuine limitation of chunk-based retrieval for document corpora.

GOG differs from GraphRAG in domain, graph construction method, and purpose. GraphRAG's graphs are constructed by an LLM reasoning over text content; GOG's graphs are constructed deterministically from static analysis of source code. GraphRAG targets document-level global queries; GOG targets surgical, task-local code modifications. GraphRAG's graph is a semantic knowledge graph; GOG's graph is a structural dependency graph encoding architectural facts — imports, package membership, generated/handwritten classification, symbol ownership — that are not inferable from content alone.

Recent work on knowledge-graph-based code generation and graph-guided retrieval for repository-level tasks shares GOG's intuition that graph structure improves over flat retrieval for code tasks. These systems focus primarily on retrieval quality — improving which files are selected. GOG's contribution includes retrieval quality but extends it: the orientation delivered to the assistant, including edit policy classification and hazard guidance, is distinct from selection alone.

2.3 Agentic Coding Systems and Repository Exploration

SWE-bench (Jimenez et al., 2024) established the dominant benchmark paradigm for repository-level code generation: given a real GitHub issue and a codebase, generate a patch that resolves it, validated by the repository's existing test suite. Systems evaluated on SWE-bench — SWE-agent, OpenHands, AutoCodeRover, Agentless, and others — demonstrate a range of approaches to repository navigation, from command-line interfaces to AST-based search to localization-repair pipelines.

A consistent pattern across high-performing SWE-bench systems is the use of repository localization as a distinct phase: identify the relevant files before attempting repair. AutoCodeRover (Zhang et al., 2024)

combines LLMs with AST-based search explicitly for this purpose. Agentless (Xia et al., 2024) uses a structured localization step before patch generation. These systems validate the intuition that knowing where to work is prerequisite to working correctly.

GOG extends this intuition in two ways. First, GOG's localization is persistent: the repository is onboarded once and the symbolic state is reused across sessions, rather than re-derived per task. Second, GOG's orientation is richer than file selection: the brief includes edit policy classification, structural anchors, verified symbol capabilities, and task-local hazard guidance — not only a list of relevant files.

The tool call reduction observed in Section 4 (38 → 14 calls) suggests that the cost of localization in current agentic systems is substantially borne by the model at inference time, through exploration. GOG proposes moving that cost upstream, to a deterministic pre-generation layer, where it can be paid once per repository onboarding rather than once per task invocation.

2.4 Static Analysis and Language Server Protocols

Static analysis tools — tree-sitter, Language Server Protocols (LSPs), Rust's compiler, TypeScript's type checker — excel at deterministic structural understanding of source code. They can identify undefined variables, unresolved imports, and type mismatches in milliseconds. What they cannot do is translate a natural language intent into a selection of relevant files, or communicate structural hazards to a language model about to generate code.

GOG occupies the gap between these two worlds. It uses deterministic static analysis for repository understanding — parsing imports, detecting generated files, classifying package strata — and translates the results into structured natural-language orientation for a language model. It does not attempt to replace static analysis; it uses static analysis as its evidence base and delivers that evidence in a form the model can act on.

2.5 The Specific Gap GOG Addresses

The systems described above share a common architecture: a language model navigates and generates within a repository, with retrieval or tool access helping it find relevant context. The orientation problem — ensuring the model begins in the right place with the right structural understanding before exploration begins — has received less attention as a distinct system layer.

GOG's hypothesis is that pre-generation orientation is a separable, automatable concern: that a deterministic system, given persistent symbolic knowledge of a repository, can reduce the model's need to explore before generating, and that this reduction has measurable effects on outcome quality, token consumption, and navigation behavior. The benchmark results in Section 4 provide early evidence for this hypothesis on a single production repository. Generalization across repositories and task families remains the primary open question.

3. System Architecture: The GOG Orientation Pipeline

GOG is not a coding assistant and not a retrieval system. It is a repo-intelligence layer that operates before a coding assistant begins generating — mapping a repository's structure, classifying its surfaces, selecting a bounded context strategy, and delivering a structured brief that orients the assistant before it touches a single file.

This section describes the five-stage pipeline that produces that orientation. The pipeline is designed around one governing constraint: every token an LLM spends discovering what it could have been told is a token wasted. GOG's job is to eliminate as much of that discovery work as possible before generation begins.

The work described here grew out of a broader theoretical program — the Symbolic Reasoning Membrane (SRM) — which investigates whether language models can be systematically demoted from architectural reasoners to linguistic renderers, receiving deterministic structured specifications rather than open-ended prompts. GOG is the applied instantiation of that thesis in the software repository domain. The SRM theoretical foundations are treated separately in a companion paper; this paper focuses on the empirical pipeline and its measured effects on coding assistant behavior.

3.1 Stage 1: Repository Onboarding

Before GOG can orient any assistant, the repository must be onboarded into persistent symbolic state. This is a one-time upfront cost — not repeated on every prompt — and it is what separates GOG from ad hoc retrieval. A repository that has been onboarded has a GOG-resident understanding of its own structure. A repository that has not is just a directory.

Onboarding proceeds in four steps. First, inventory: the pipeline walks the repository tree, excluding known build, vendor, cache, and generated directories, recording language, inferred role, package membership, path tokens, and whether the file is mechanically produced. Second, parse and graph construction: supported languages are parsed into a structural dependency graph with language-neutral schema. Third, constraint and convention extraction: TypeScript path aliases, test file naming patterns, generated API surfaces, and package manifest relationships are extracted as queryable facts. Fourth, artifact persistence: all artifacts are written to a .gog/ directory and reused across sessions, with a freshness check detecting graph drift from the working tree.

3.2 Stage 2: Task-Fit Analysis and Context Strategy Selection

When a prompt arrives, GOG first asks a structural question: how does this task relate to the repository graph we already have? The task-fit analyzer extracts concepts from the prompt and attempts to map them to graph nodes, producing metrics including prompt-graph fit, language fit, stratum confidence, novelty burden, generated/noise risk, and contradiction signals.

These metrics feed a context strategy decision that selects one of five serving modes before any files are retrieved:

3.3 Stage 3: Graph-Routed Context Selection

With a strategy selected, GOG executes context selection. For the graph_membrane path — the primary mode for well-scoped tasks — seed nodes are resolved from explicit file references or named symbols. The graph is then traversed outward to a controlled depth, capturing the immediate structural neighborhood. Candidate files are scored against keyword relevance, graph distance, stratum alignment, role appropriateness, and token budget constraints. The membrane records both kept and rejected files with explicit reasons.

Every selected file passes through a rule-based classifier that determines its edit status: default_edit_allowed, evidence_only, or high_risk. This classification is fully automated based on path analysis — files in /generated/, /vendor/, /node_modules/, /dist/ are tagged without per-task configuration. The model receives this classification as part of the brief.

3.4 Stage 4: Brief Synthesis

The context bundle is rendered into a structured Markdown assistant brief containing: the task prompt verbatim; selected files with language, role, token estimate, generated status, and edit permission; file edit policy summary with explicit lists of edit targets and evidence-only files; context strategy decision and its supporting metrics; structural anchors showing graph edges between selected files; verified symbol

capabilities; mutation constraints including expected edit files and forbidden globs; and the validation command.

One component requires explicit disclosure: task-level notes under Mutation Constraints may include human-authored semantic hazard guidance. In the current system, this guidance is authored by a researcher who understands the repository. The GOG graph encodes the structural evidence from which such warnings could in principle be synthesized automatically — generated files are tagged, stratum separations are explicit, adjacency between generated and handwritten surfaces is a graph property — but the synthesis of natural-language hazard warnings from that structural evidence is an active research target, not a current capability. Section 4.7 addresses the implications of this distinction for interpreting the benchmark results.

3.5 Stage 5: Integration with Coding Assistants

GOG is implemented as a preflight layer, not a replacement for existing coding assistants. The opencode-gog integration demonstrates this concretely: a thin wrapper around OpenCode that generates a GOG or RAG brief, writes it to a timestamped artifact directory, and invokes the coding assistant with the brief attached via --file. The wrapper supports three modes — --gog, --rag, and --baseline — enabling direct controlled comparison. The integration architecture is intentionally shallow: no changes to the coding assistant's tool calling, model weights, or internal context management are required.

3.6 The Reasoner Slot

The architecture as described positions GOG as the substrate and a general-purpose language model as both the reasoner and renderer. The longer-term architectural thesis — inherited from the SRM research program — is that reasoning and rendering should be separated more explicitly: a deterministic or symbolic reasoning layer consumes GOG's structured repo state and produces a typed mutation plan, which a small language model then renders into syntax.

Early SRM experiments show that a 500M parameter model given a deterministic symbolic specification produces correct structured code that failed entirely under raw prompting — an 83.6% wall-clock reduction and 88.1% token reduction compared to RAG prompting on the same task. This result motivates the architecture even if the full symbolic reasoner remains a research target. For the purposes of this paper, the reasoner slot is occupied by the coding assistant model.

4. Empirical Evaluation: Orientation as a Measurable Property of Context

The central claim of this paper is that pre-generation orientation produces measurably different model behavior than retrieval alone. This section presents controlled benchmark evidence from a real-world software repository across three experimental layers, culminating in the first native integration result using the opencode-gog adapter.

All benchmarks were conducted against the Orca Whirlpools SDK, a production-grade open-source repository implementing concentrated liquidity market maker (CLMM) protocols across TypeScript and Rust. The repository contains generated account types, handwritten public helper APIs, cross-language deployment constants, and deep stratum separation — properties that expose orientation failures in ways synthetic benchmarks cannot.

4.1 Experimental Design

Three context modes were evaluated. Baseline: the coding assistant with standard tool use and no context pre-processing. RAG Hybrid: a retrieval-augmented context bundle assembled using a hybrid 16K-token budget. GOG: a structured assistant brief produced by the GOG Professional context engine, comprising surgical file selection, explicit edit and evidence boundaries, and task-level semantic hazard notes assembled from the repository's onboarded symbolic state and a human-curated task definition.

Each mode ran against the same repository commit, task prompt, model, and validation commands. Results were evaluated against executable validation using the repository's own test and typecheck infrastructure.

4.2 The Benchmark Task: A Semantic Trap in TypeScript

The primary task crosses a hazard common in real codebases: the generated-versus-consolidated type trap. The Orca Whirlpools SDK maintains both generated account types (automatically produced by an IDL compiler) and handwritten consolidated helper types (the correct public API surface). A naively oriented model will tend to reach for the generated DynamicTickArray type — it is prominent, keyword-matched, and syntactically plausible. But the repository's type system rejects it: Account<DynamicTickArray & {{ __kind: "Dynamic" }}, TAddress> is not assignable to Account<TickArray, TAddress>. This failure passes Vitest (runtime tests) and fails only at tsc --noEmit (static typecheck).

Task prompt: In the TypeScript client tick-array state helper, add exported type guard helpers isFixedTickArrayAccount and isDynamicTickArrayAccount for decoded consolidated tick-array accounts. They should inspect the consolidated data.__kind field and narrow to the corresponding account kind. Add tests using the existing fixed and dynamic tick-array fixtures in tickArray.test.ts.

Validation: cd ts-sdk/client && vitest run tests && tsc --noEmit

4.3 Three Layers of Reproducibility

Layer 1 — Manual benchmark harness (prior work): Baseline failed. GOG v1 selected correct files but lacked hazard guidance and failed. RAG passed. GOG v2, with bad-path guidance added to the task definition, passed. Criticism addressed: the harness itself might have created the result.

Layer 2 — Repeatable controlled bakeoff (prior work): Re-run with controlled harness, identical worktrees, and audited fair RAG baseline. Results consistent across runs. Criticism addressed: single-run artifact.

Layer 3 — Native opencode-gog adapter (this paper): The opencode-gog binary used as a native coding assistant command, generating GOG and RAG briefs automatically via OpenCode's --file interface. No benchmark-specific code, no manual brief attachment, no retrieval changes, no adapter modifications during the run. Criticism addressed: semantic-hazard guidance was an artifact of the internal harness, not a real capability.

4.4 Results

4.5 The Orientation Signal: Tool Call Reduction

Token reduction is a consequence of better orientation, but it is not the primary finding. The most informative metric is tool calls. Baseline consumed 38 tool calls and 17 search calls before producing a result that failed typecheck. GOG consumed 14 tool calls and 2 search calls — a 63% reduction in tool calls and 88% reduction in search calls — before producing a result that passed both.

Tool calls represent exploration behavior: file reads, directory listings, symbol searches, diagnostic probes. A model requiring 38 tool calls is engaged in extensive repository archaeology. A model requiring 14 has

received enough orientation that it begins in approximately the right place. The baseline model's 17 search calls produced 18 wrong-stratum file reads — it found something plausible and it was wrong. GOG's 2 search calls produced 2 wrong-stratum reads. The brief had named the trap explicitly.

4.6 Context Efficiency: The Brief Comparison

GOG's assistant brief was 9,404 characters. RAG's was 24,988 characters — 2.65× larger. Both passed. This supports the claim that context quality is a stronger determinant of outcome than context quantity, at least for tasks where the primary failure mode is semantic navigation rather than information retrieval.

RAG's larger brief contained correct files and edit boundaries, plus wrong-stratum files selected by semantic similarity. The model resolved this successfully — but at the cost of greater exploration (20 tool calls vs. 14) and a larger transcript (471,757 tokens vs. 196,008). RAG's noise tolerance succeeded here; on a harder task or under a tighter budget, that tolerance may not hold.

4.7 Honest Scope: What This Result Does and Does Not Show

Single repository family. All benchmarks were conducted against the Orca Whirlpools SDK. Results do not generalize by construction to other repositories, languages, or task families.

RAG also passed. The claim is that GOG achieves equivalent validated outcomes with fewer tool calls, less transcript overhead, and a more precise context bundle. RAG is a competitive and capable baseline.

The semantic hazard guidance has two distinct components. The structural classification — generated file detection, edit boundary labeling, evidence-only tagging — is fully automated via repo_cartography.py and file_policy.py, with no per-task configuration and no Orca-specific identifiers in the code. The natural-language hazard warning included in the brief was authored by a human in the task definition's notes field. The graph encodes all necessary evidence for automated synthesis — generated files tagged, strata separated, discriminant patterns detected — but the synthesis of actionable natural-language guidance from that evidence is an active research target. A forthcoming synthetic regression benchmark will test whether the structural classification alone, without task-authored notes, reproduces the tool call reduction on a structurally equivalent but non-Orca repository.

The current system is appropriately characterized as Level 2 of three: structural detection and surface classification are automated and generalizable; natural-language hazard articulation is task-authored and a current research target.

Wall-clock favors GOG but must be interpreted carefully. Wall-clock includes model inference time, network latency, and tool execution. Brief generation overhead is not separately instrumented and is future work.

4.8 Selected File Comparison

GOG selected 6 files: package.json, dynamicTickArray.ts (generated, evidence only), gpa/dynamicTickArray.ts (evidence only), state/dynamicTickArray.ts (evidence only), state/tickArray.ts (primary edit surface), and tickArray.test.ts (test surface). The evidence-only labels on generated files were produced automatically by file_policy based solely on path analysis.

RAG selected 10+ files including additionally: programs/whirlpool/src/state/tick_array.rs (wrong stratum, Rust), legacy-sdk/whirlpool/src/utils/remaining-accounts-util.ts (wrong stratum, legacy), and others. RAG included the same generated files without the edit/evidence distinction, leaving the model to infer the

boundary from content alone. The boundary label is not retrievable by similarity search because the distinction between an evidence file and an edit surface is a structural property of the repository's architecture, not a semantic property of file content.

5. Limitations and Future Work

5.1 Single Repository Family

All benchmark results were produced against the Orca Whirlpools SDK. The immediate next milestone is replication across at least four structurally distinct repositories: a large TypeScript monorepo without the generated-vs-consolidated pattern, a Python web framework codebase, a Rust systems library, and a cross-language repository outside the Solana ecosystem.

5.2 Single Task Family

The primary benchmark task belongs to the TypeScript type system navigation family. Four task families are identified as high-priority for future replication: type system tasks (demonstrated here), cross-language parity tasks (partially addressed in prior GOG work), architecture navigation tasks spanning five to ten files, and bug-fixing tasks with unknown root causes. The bug-fixing family is hypothesized to show the largest GOG advantage, because root cause localization requires the most repository exploration — and orientation should reduce that exploration cost most dramatically.

5.3 The Natural-Language Hazard Synthesis Gap

As documented in Section 4.7, GOG's structural classification is fully automated. The natural-language hazard warning was authored by a human in the task definition. The validation methodology for automated synthesis is a synthetic regression benchmark: a minimal repository with the same structural pattern — generated account types adjacent to handwritten consolidated helpers, discriminant fields — but no Orca-specific identifiers. If GOG's structural detection produces equivalent edit boundary classification and tool call reduction on the synthetic repository without task-authored notes, the generalization claim is supported.

5.4 Wall-Clock and Brief Generation Overhead

Brief generation time — loading onboarding artifacts, running context strategy selection, executing graph search, classifying file policies, and rendering the Markdown brief — is not separately instrumented in the current benchmark. For the orientation hypothesis to hold economically, this overhead must be substantially smaller than the tool call savings it produces. Future work should instrument brief generation separately and report it as a distinct cost component.

5.5 The Reasoner Slot: Toward Symbolic Hazard Synthesis

The path from the current GOG system to the SRM architecture runs directly through automated hazard synthesis: once GOG can synthesize structured warnings from graph evidence without human authorship, the distance between an orientation brief and a typed mutation plan shortens considerably. Future work will pursue this path explicitly, treating hazard synthesis as the bridge between the current system and the longer-term SRM architecture.

5.6 Multi-Model and Multi-Assistant Evaluation

All benchmark results used a single model (ollama/kimi-k2.6:cloud) through a single assistant interface (OpenCode). GOG's orientation is model-agnostic in principle — the brief is a Markdown document — but

whether larger or smaller models benefit proportionally from orientation has not been measured. The hypothesis is that smaller models benefit more, having less capacity for repository archaeology. Systematic evaluation across model sizes and families is future work.

6. Conclusion

Software repositories are not flat bags of text. They are graphs: packages, files, imports, symbols, generated surfaces, handwritten helpers, test infrastructure, validation commands, and architectural strata. More than that — for a repository to function at runtime, it must compile. The compiler is the ultimate proof that a codebase is not loosely organized prose. It is rigidly structured. Import paths must resolve. Types must align. Package boundaries must be respected. Every constraint the compiler enforces is a constraint GOG can reason about deterministically, before a language model spends a single token doing it probabilistically.

Coding assistants that approach repositories as flat bags of text spend tokens discovering structure that was always there, deterministically knowable, and expressible without a language model. GOG's thesis is that this structure should be precomputed, persisted, and delivered — so the assistant can spend its budget on generating, not on navigating.

GOG is an attempt to build that deterministic layer and connect it to the coding assistants that currently work without it. The system onboards a repository into persistent symbolic state, selects a context strategy based on structural signals, classifies every file as an edit target or evidence surface, and delivers a compact structured brief to the assistant before generation begins.

The benchmark result that motivated this paper is simple to state: on a controlled task against a real production repository, an assistant given a GOG brief consumed 14 tool calls to produce a validated result. The same assistant with no brief consumed 38 tool calls and produced a result that failed static typecheck. The model did not reason differently. It navigated differently — because it knew where it was before it started moving.

This is a single result on a single repository. The honest interpretation is narrow: pre-generation orientation demonstrably changes model navigation behavior on this task, and the structural classification that makes it possible is automated and generalizable in principle. Whether it holds broadly across repositories, task families, and model sizes is the question this paper opens rather than closes.

The thesis that motivated GOG — and the broader SRM research program from which it emerged — is that language models are most powerful when they are not asked to do things that deterministic systems can do better. Repository navigation, stratum classification, edit boundary detection, and hazard identification are deterministic problems. Code generation, from a well-oriented starting point, is a linguistic problem. Keeping those concerns separated, and building the infrastructure that makes the separation real, is the research agenda this paper advances.

GOG is infrastructure for coding assistants. This paper is the first systematic evidence that such infrastructure changes what assistants do.

Mode	Validation	Typecheck	Wall clock	Tool calls	Search calls	Transcript tokens	Brief chars
Baseline	FAIL	FAIL	573.62s	38	17	626,497	—
GOG	PASS	PASS	349.91s	14	2	196,008	9,404
RAG	PASS	PASS	469.91s	20	0	471,757	24,988

Table 1. Model: ollama/kimi-k2.6:cloud. Same repository commit, task prompt, and validation commands across all three modes.

Strategy	Condition	Behavior
`graph_membrane`	High fit, high stratum confidence, low novelty	Serve tight graph-routed context directly
`stratum_expand`	Moderate confidence, over-pruning risk	Expand to full package stratum
`layered_extension`	Task spans multiple architectural layers	Serve per-layer context with provenance
`hybrid_fallback`	Weak prompt-graph fit	Fall back to hybrid retrieval
`clarify_or_impossible`	Contradiction signals present	Return structured preflight; spend no tokens

Table 2. Context strategy decision modes.

Acknowledgments and AI Assistance Declaration

The author acknowledges the use of frontier language model assistants (Claude and ChatGPT) as architectural sounding boards, draft reviewers, and research collaborators during the preparation of this manuscript and the development of the GOG system. All research design, experimental methodology, benchmark execution, system architecture, and conclusions are the original work of the author.

GOG Lite, the public reference implementation, is available at https://github.com/dchisholm125/graph-oriented-generation. GOG Professional, the private research engine underlying the benchmark results in this paper, is not publicly released. Researchers interested in collaboration on the semantic hazard synthesis capability should contact the author directly.

References

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. Microsoft Research. arXiv:2404.16130.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.

Zhang, Y., Zhang, F., Blohm, J., Ahmad, B., Tian, H., & Keung, P. (2024). *AutoCodeRover: Autonomous Program Improvement*. arXiv:2404.05427.

Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2024). *Agentless: Demystifying LLM-based Software Engineering Agents*. arXiv:2407.01489.

Gao, Y., Xiong, Y., Gao, X., et al. (2023). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997.

Yang, J., Jimenez, C. E., Wettig, A., et al. (2024). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. arXiv:2405.15793.

Chisholm, D. R. (2025). *Graph-Oriented Generation (GOG): Offloading AI Reasoning to Deterministic Symbolic Graphs*. Independent preprint. https://github.com/dchisholm125/graph-oriented-generation.